

Manos a la Obra

Documentación final del proyecto, explicada por partes, para estudiar el código y defenderlo en el examen.

Práctica Profesionalizante · Analista Funcional en Sistemas · Versión final

1	Visión general del proyecto	qué hace, usuarios, archivos
2	HTML: las páginas	estructura y formularios
3	CSS: los estilos	style.css por partes
4	Base de datos	las 4 tablas y sus relaciones
5	Conexión entre PHP y la base	qué hace cada archivo
6	Relevamiento y pruebas	errores encontrados y corregidos
7	Puesta en marcha	XAMPP, SQLyog, mails, errores comunes
8	Preparar la defensa	guion, preguntas, glosario

1 Visión general

Qué hace el sitio, para quién y con qué está construido.

Manos a la Obra es un sitio web que conecta a vecinos que necesitan un arreglo en su casa con **profesionales de oficios** (electricistas, plomeros, gasistas, pintores, etc.) de Rosario. Los profesionales publican su perfil; los clientes buscan por oficio, palabra o barrio, leen las reseñas de otros clientes, llaman directo al profesional y después pueden dejar su propia reseña.

Los tres tipos de usuario

Visitante (sin cuenta)

Busca, filtra por oficio o barrio, ve perfiles y reseñas, escribe por Contacto.

Cliente

Todo lo del visitante, y además deja **una reseña** por profesional (1 a 5 estrellas).

Profesional

Completa y edita su perfil. Aparece en el inicio. Puede contratar el plan **Destacado**.

El sitio cambia según quién mira: los botones del encabezado, el aviso "Completá tu perfil", la página "Cómo funciona" y el formulario de reseña muestran cosas distintas para cada uno.

Las cuatro capas del proyecto

HTML

Define **qué** hay en cada página: títulos, tarjetas, formularios.

CSS

Define **cómo se ve**: colores, tamaños, ubicación, modo oscuro, celular.

PHP

Corre en el servidor: lee y guarda datos, valida, maneja la sesión, manda mails.

MySQL

La base de datos: clientes, profesionales, reseñas y barrios.

Herramientas de trabajo: **XAMPP** (prende Apache, que ejecuta PHP, y MySQL), **SQLyog** (para ver y editar la base), el **navegador** (se entra por `http://localhost/manos_a_la_obra_con_css/`) y **Git/GitHub** (historial de cambios compartido por el grupo). No se usa ningún framework ni librería: todo está escrito a mano.

Funcionalidades

Funcionalidad	Dónde está
Registro e inicio de sesión	`registro.php` → `guardar.php`, `login.php`, `logout.php`
Perfil del profesional	`postular.php` (completar / editar)
Buscador y filtro por oficio	`index.php` (consulta con `LIKE`)
Profesionales cerca (por barrio y zona)	`index.php` + tabla `barrios`
Perfil público y reseñas	`perfil.php` + tabla `resenas`

Cómo funciona (según el usuario)	`como_funciona.php`
Contacto con imágenes por mail	`contacto.php` (función `mail()`)
Modo oscuro	`tema.php` (cookie) + variables de `style.css`
Plan Destacado (monetización)	columna `destacado_hasta`, se activa desde SQLyog

Archivos del proyecto

Archivo	Para qué sirve
`index.php`	Inicio: buscador, oficios, "Profesionales cerca" y la grilla de profesionales.
`tarjeta_profesional.php`	No se abre sola. Es el HTML de una tarjeta; `index.php` la incluye en sus dos grillas.
`perfil.php`	Perfil público de un profesional, sus reseñas y el formulario para dejar una.
`como_funciona.php`	Explica el sitio según quién mira (visitante, cliente o profesional).
`contacto.php`	Formulario que manda un mail con hasta 3 imágenes a iseiclaude@gmail.com.
`registro.php`	Formulario para crear una cuenta.
`guardar.php`	No se ve. Valida el registro y guarda la cuenta en `clientes` o `profesionales`.
`login.php` / `logout.php`	Iniciar y cerrar sesión.
`postular.php`	El profesional completa o edita su perfil.
`tema.php`	No se ve. Cambia entre modo claro y oscuro y vuelve a la página anterior.
`config.php`	Conexión a la base. Lo incluyen todas las páginas que usan la base.
`style.css`	Todos los estilos del sitio, con un índice de 16 partes al principio.
`database.sql`	Crea la base, las 4 tablas y los datos de ejemplo. Se ejecuta en SQLyog.
`README.md` / `CLAUDE.md`	Documentación del repositorio en GitHub.

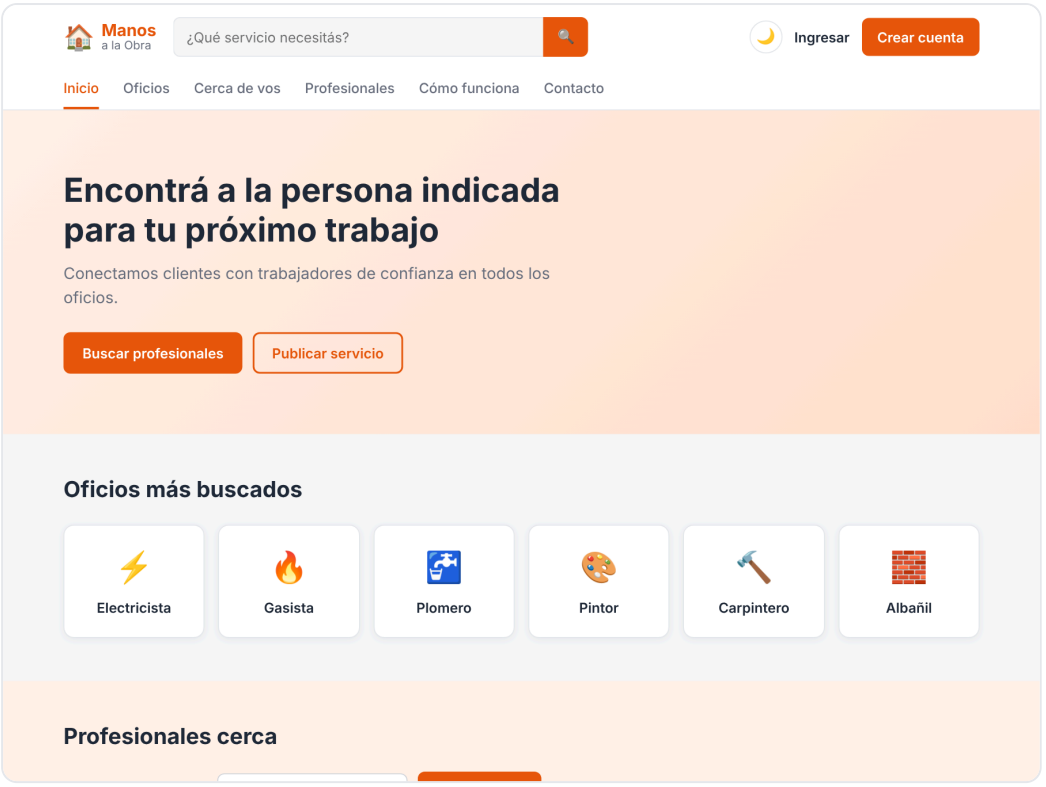
Recorridos

Profesional



Cliente

index.php Busca, filtra o elige su barrio → **perfil.php** Lee datos y reseñas, llama → **login.php** Ingresa → **perfil.php** Deja su reseña → **index.php** La valoración se actualizó



La página de inicio.

2 HTML: las páginas

La estructura y el contenido de cada pantalla.

HTML y PHP en el mismo archivo

Las páginas son `.php` y no `.html` porque mezclan HTML con partes de PHP. El servidor ejecuta primero el PHP (por ejemplo, busca los profesionales en la base) y al navegador le llega HTML común. Todas las páginas siguen el mismo orden: **arriba, en PHP, se buscan y validan los datos; abajo viene el HTML que los muestra.**

```
<!— Lo que está escrito en el archivo —>
<h3><?php echo htmlspecialchars($profesional["nombre"]); ?></h3>
<!— Lo que recibe el navegador —>
<h3>María Gómez</h3>
```

Para las condiciones y los ciclos dentro del HTML se usa la sintaxis con dos puntos, que se lee mejor entre etiquetas: `if (...): ... else: ... endif;`, `foreach (...): ... endforeach;` y `while (...): ... endwhile;`.

Estructura básica de cada página

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">                  <!— tildes, ñ y emojis —>
    <meta name="viewport" content="width=device-width, initial-scale=1.0">  <!— celular —>
    <title>Manos a la Obra</title>
    <link rel="stylesheet" href="style.css">   <!— los estilos —>
</head>
<body class="<?php echo $modo_oscuro ? "oscuro" : ""; ?>">   <!— modo oscuro —>
    <header> ... </header>   <main> o <section> ... </section>   <footer> ... </footer>
</body>
</html>
```

Se usan etiquetas **semánticas**: `<header>` (encabezado), `<nav>` (menú), `<section>` (cada parte del inicio), `<main>` (contenido principal) y `<footer>` (pie). Le dicen al navegador y a los lectores de pantalla qué es cada parte.

index.php: el inicio, de arriba hacia abajo

Parte	Qué contiene
`<header>`	Logo, buscador, botón 🌙/☀️, botones de cuenta y menú (`<nav class="barra-menu">`). Los botones cambian según haya sesión.
Aviso de perfil	Solo para un profesional sin perfil: "Completá tu perfil para aparecer en el inicio".
`.hero` (portada)	Título grande y los botones "Buscar profesionales" y "Publicar servicio" (este último no aparece para clientes).

Oficios más buscados	Seis tarjetas que son links: cada una filtra por su oficio.
Profesionales cerca	Una lista de barrios; al elegir uno, muestra profesionales de la misma zona.
Profesionales destacados	Una tarjeta por profesional. El título cambia con la búsqueda o el filtro.
`<footer>`	Puntos de confianza y botón "Consultanos" (lleva a Contacto).

Mostrar cosas distintas según quién mira

```
<?php if (isset($_SESSION["usuario_nombre"])): ?>
    <span class="link-ingresar">Hola, <?php echo htmlspecialchars($_SESSION["usuario_nombre"]); ?
></span>
    <?php if (($_SESSION["usuario_tipo"] ?? "") === "profesional"): ?>
        <a href="postular.php" class="btn-publicar">Postular servicio</a>
    <?php endif; ?>
    <a href="logout.php" class="btn-publicar">Salir</a>
<?php else: ?>
    <a href="login.php" class="link-ingresar">Ingresar</a>
    <a href="registro.php" class="btn-publicar">Crear cuenta</a>
<?php endif; ?>
```

El buscador

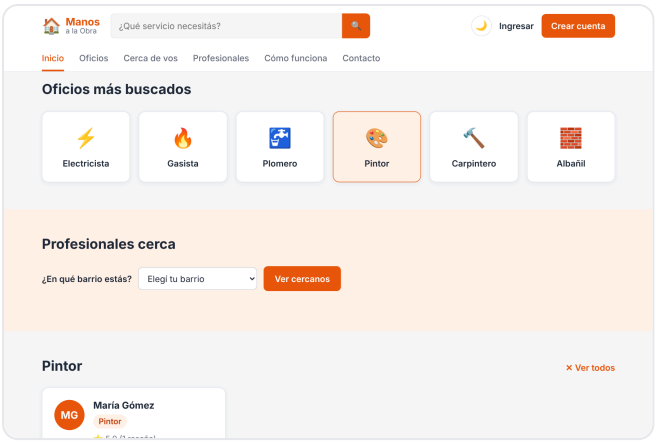
Es un `<form>` con `method="get"`: lo buscado viaja **en la dirección** (`index.php?buscar=pint`), así el resultado se puede compartir o guardar en favoritos. El `value` vuelve a mostrar lo buscado y `#profesionales` hace bajar directo a los resultados.

```
<form class="buscador" action="index.php#profesionales" method="get">
    <input type="search" name="buscar" placeholder="¿Qué servicio necesitás?"
           value="<?php echo htmlspecialchars($buscar); ?>">
    <button type="submit" aria-label="Buscar">🔍</button>
</form>
```

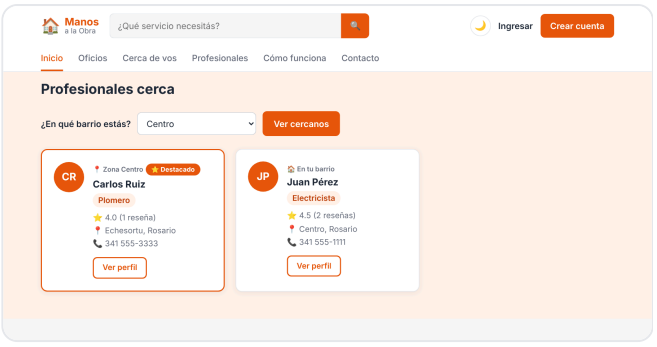
Oficios: tarjetas generadas con foreach

Los seis oficios están en un array de PHP (`"Pintor" ⇒ "🎨"`) y un `foreach` arma una tarjeta por cada uno, en lugar de repetir el HTML seis veces. `urlencode()` arma bien la dirección aunque el oficio tenga tildes o ñ (Albañil). La tarjeta del oficio elegido recibe la clase `elegido`.

```
<?php foreach ($oficios_destacados as $nombre_oficio ⇒ $icono): ?>
    <a href="index.php?oficio=<?php echo urlencode($nombre_oficio); ?>#profesionales"
       class="oficio-card <?php echo $oficio === $nombre_oficio ? "elegido" : ""; ?>">
        <span class="icono"><?php echo $icono; ?></span>
        <p><?php echo $nombre_oficio; ?></p>
    </a>
<?php endforeach; ?>
```



Filtro por "Pintor": la tarjeta queda marcada y aparece "× Ver todos".



"Profesionales cerca" con el barrio Centro elegido.

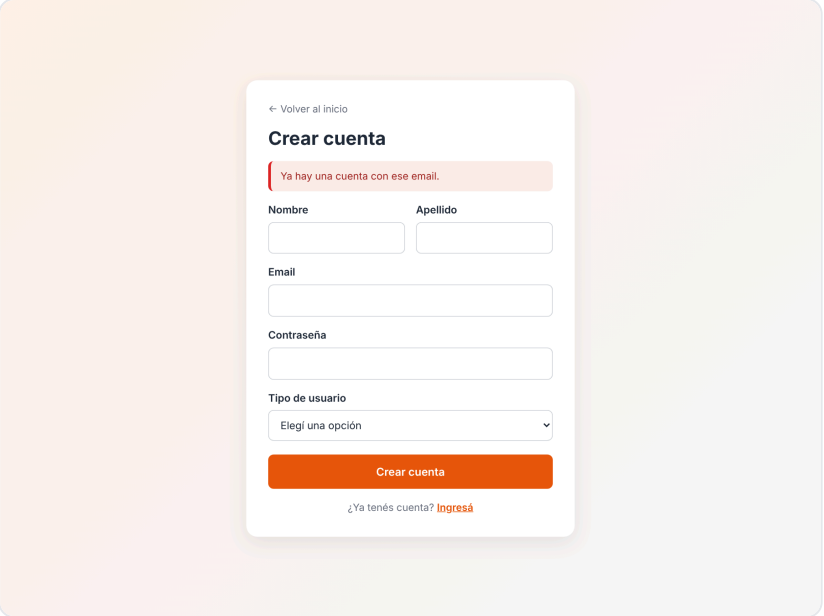
Una tarjeta, dos grillas: include

La tarjeta de un profesional se usa en dos lugares ("Profesionales cerca" y la grilla principal). Para no repetir el HTML, está en su propio archivo, `tarjeta_profesional.php`, y el inicio lo incluye dentro del ciclo:

```
<?php while ($profesional = $resultado_profesionales->fetch_assoc()): ?>
    <?php
    $etiqueta = "";   // en esta grilla las tarjetas no llevan etiqueta
    include "tarjeta_profesional.php";
    ?>
<?php endwhile; ?>
```

Antes de incluirla, la página prepara dos variables: `$profesional` (la fila) y `$etiqueta` ("🏠 En tu barrio", "📍 Zona Centro" o vacío). Si alguien abre `tarjeta_profesional.php` directo en el navegador, el archivo se corta con `exit` porque no hay profesional para mostrar.

Los formularios



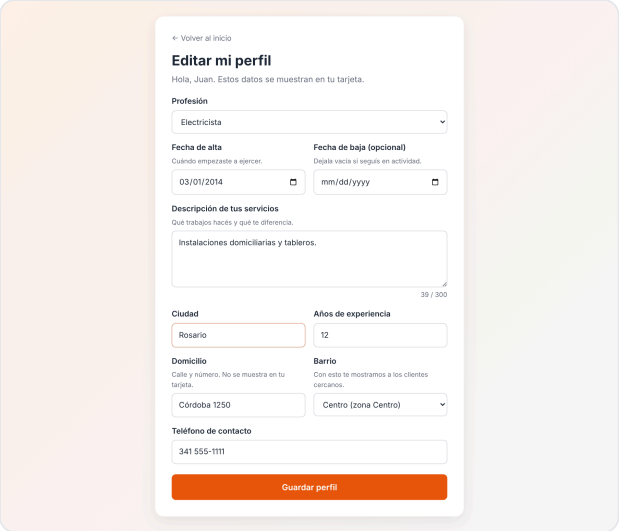
Crear cuenta mostrando un error.

Elemento	Qué hace
`<form action="..." method="post">`	`action` es el archivo PHP que recibe los datos. `post` los manda ocultos.
`name="email"`	El nombre con el que PHP lee el campo: `$_POST["email"]`. Sin `name`, el dato no se envía.
`<label for="email">` + `id="email"`	Une el texto con su campo: al tocar el texto se activa el campo, y los lectores de pantalla saben qué es.
`required`, `maxlength`, `min`, `max`	Validaciones del navegador: obligatorio, largo máximo, número o fecha mínima/máxima.
`type="email"`, `"password"`, `"date"`, `"number"`, `"tel"`, `"search"`, `"file"`	Tipos de campo: validan el email, ocultan la contraseña, muestran un calendario, solo aceptan números, etc.
`<select>` + `<option>`	Lista desplegable (tipo de usuario, profesión, barrio, puntaje, asunto).
`<textarea maxlength="300">`	Texto largo con máximo de caracteres.

Las validaciones del navegador no alcanzan: cualquiera puede saltearlas (por ejemplo, editando la página). Por eso **PHP vuelve a validar todo en el servidor**. Las del navegador están para ayudar al usuario; las de PHP, para proteger el sistema.

GET o POST: con `get` los datos van en la dirección y se ven; se usa para buscar, filtrar y elegir barrio. Con `post` van ocultos; se usa para todo lo que guarda datos o lleva contraseñas: registro, login, perfil, reseña y contacto.

postular.php: el perfil del profesional

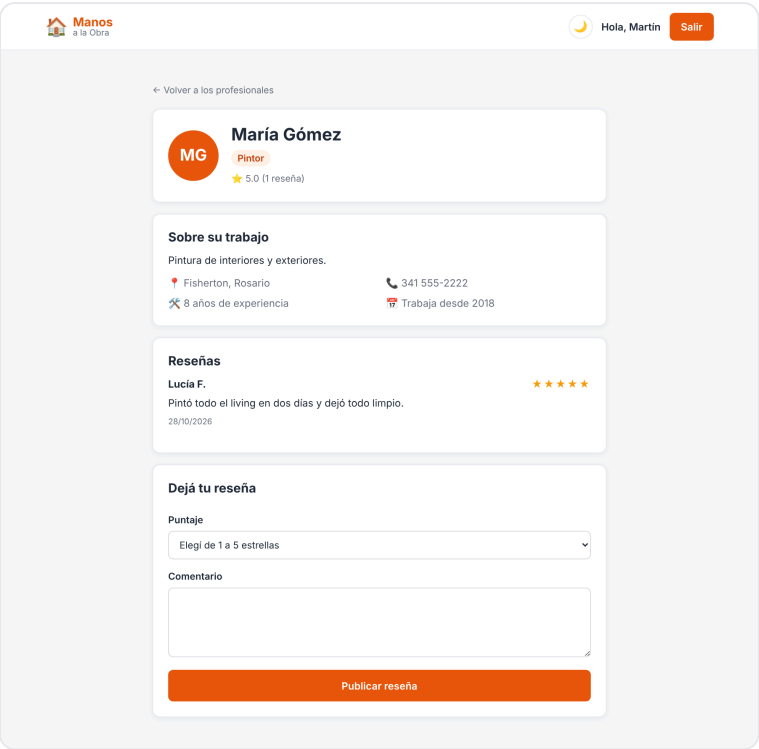


El campo "Ciudad" tiene el foco y se marca en naranja.

- Si ya tiene perfil, los campos aparecen **completos** con sus datos para editarlos. Si hubo un error, muestran lo que escribió.
- La fecha de alta tiene `max` con la fecha de hoy: el calendario no deja elegir una fecha futura.
- El domicilio se pide pero **nunca se muestra** en la página; solo el barrio y la ciudad.
- Tiene el **único JavaScript** del proyecto: un contador que muestra "120 / 300" mientras se escribe la descripción.

perfil.php: el perfil público

Se abre desde "Ver perfil", con el `id` en la dirección: `perfil.php?id=2`.



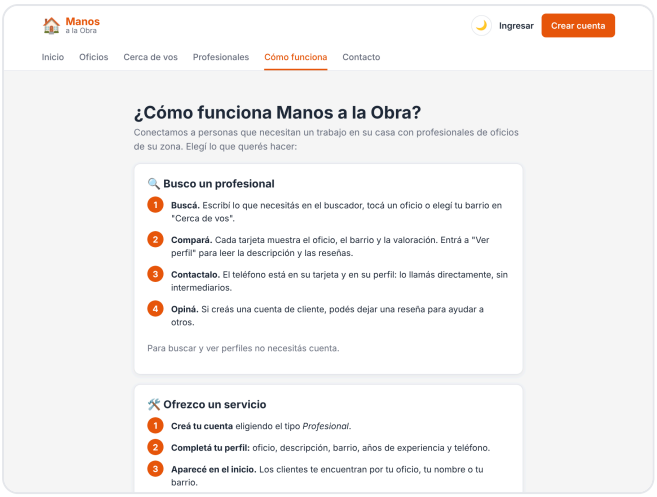
Perfil visto por un cliente: aparece el formulario de reseña.

Quién mira	Última caja
Sin sesión	"Ingresá con tu cuenta de cliente para dejar una reseña".
Profesional	"Solo los clientes pueden dejar reseñas".
Cliente que ya opinó	"Ya dejaste tu reseña".
Cliente que no opinó	El formulario (puntaje y comentario).

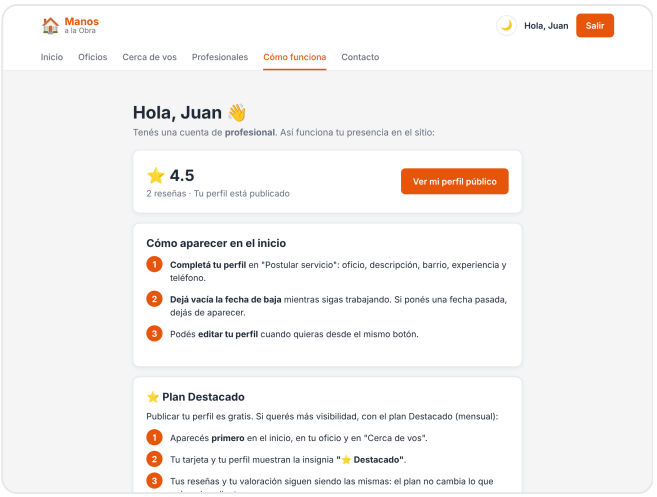
Si el `id` no existe o el profesional está dado de baja, muestra "Perfil no disponible".

como_funciona.php: una página, tres versiones

Con un `if / elseif / else` de PHP según `$_SESSION["usuario_tipo"]`, la página muestra tres contenidos distintos. El cliente ve cuántas reseñas dejó; el profesional ve el estado de su perfil (sin completar, dado de baja o publicado con su valoración) y su plan Destacado.

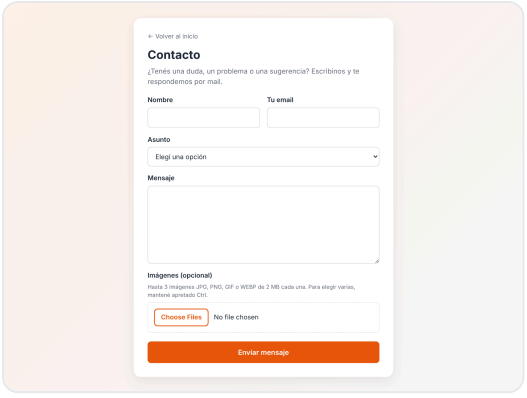


Versión para visitantes.



Versión para un profesional publicado.

contacto.php: formulario con archivos



El formulario de contacto.

```
<form action="contacto.php" method="post"
      enctype="multipart/form-data">
  ...
  <input type="file" name="imagenes[]"

accept="image/jpeg,image/png,image/gif,image/webp"
         multiple>
```

- `enctype="multipart/form-data"` es **obligatorio** para mandar archivos; sin eso solo viajan los nombres.
- `multiple` deja elegir varios archivos, y el `[]` en el `name` hace que PHP los reciba como una lista.
- `accept` hace que la ventana para elegir muestre solo imágenes.
- Si la página se abre como `contacto.php?asunto=Plan+Destacado`, el asunto ya aparece elegido.

3 CSS: los estilos

Todo el aspecto visual está en un solo archivo: style.css.

Cómo se conecta con el HTML

El HTML dice **qué** hay; el CSS dice **cómo se ve**. Todas las páginas cargan el mismo archivo con `<link rel="stylesheet" href="style.css">`, así que un cambio en `style.css` se ve en todo el sitio a la vez.

El índice de style.css

El archivo empieza con un índice. Cada parte tiene su número en un comentario, así se encuentra con Ctrl+F (por ejemplo, buscando "11.").

N°	Parte	Qué controla	Técnicas
1	Variables	Colores, bordes, redondeo, sombra.	`:root`, `--nombre`
2	Modo oscuro	Cambia el valor de las variables.	`body.oscuro`
3	Base	Letra, fondo, `.contenedor` centrado de 1100px.	`box-sizing`, `margin: 0 auto`
4	Botones	Lleno (naranja) y con borde.	`:hover`, `transition`
5	Encabezado y menú	Barra fija con logo, buscador y botones.	`sticky`, `flex`
6	Aviso de perfil	Franja naranja "Completá tu perfil".	`flex-wrap`
7	Portada (hero)	Degradado y título grande.	`linear-gradient`
8	Secciones y oficios	Espaciado y tarjetas de oficios.	`grid`, `transform`
9	Profesionales cerca	Fondo naranja clarito y lista de barrios.	`flex`
10	Plan Destacado	Borde naranja e insignia "⭐ Destacado".	dos clases juntas
11	Tarjetas de profesionales	Avatar redondo con iniciales, datos.	`grid`, `border-radius: 50%`
12	Pie de página	Fondo azul oscuro y lista en dos columnas.	`grid-template-columns`
13	Perfil	Cajas blancas, datos en dos columnas, reseñas.	`:first-of-type`
14	Cómo funciona	Pasos numerados en círculos.	`counter`, `::before`
15	Formularios	Tarjeta centrada, campos, mensajes.	`:focus`, `::file-selector-button`

16	Celulares	Adapta todo a pantallas chicas.	`@media`

Cómo se escribe una regla

```
.boton-lleno {               /* selector: a qué elementos se aplica */
    background: var(--primario); /* propiedad: valor; */
    color: var(--blanco);
}
```

En el HTML, el estilo se activa con la clase: `<a class="boton boton-lleno">`. Un elemento puede tener varias clases separadas por espacios.

Tipos de selectores que usa el proyecto

Selector	Ejemplo	A qué se aplica
De etiqueta	`body`, `a`	A todos los elementos de ese tipo.
De clase	`.oficio-card`	A los elementos con esa clase. El más usado.
Dos clases juntas	`.profesional-card.destacada`	Solo a los que tienen **las dos** clases.
Descendiente	`.formulario input`	A los `input` que están dentro de un `.formulario`.
Hijo directo	`.fila > label`	Solo a los `label` justo adentro de `.fila`.
De atributo	`input[type="file"]`	A los `input` de tipo archivo.
Pseudo-clase	`:hover`, `:focus`, `:first-of-type`	A un estado o una posición.
Pseudo-elemento	`::before`, `::file-selector-button`	A una parte "extra" del elemento (el círculo con el número, el botón "Elegir archivos").
Universal	`*`	A todos los elementos.

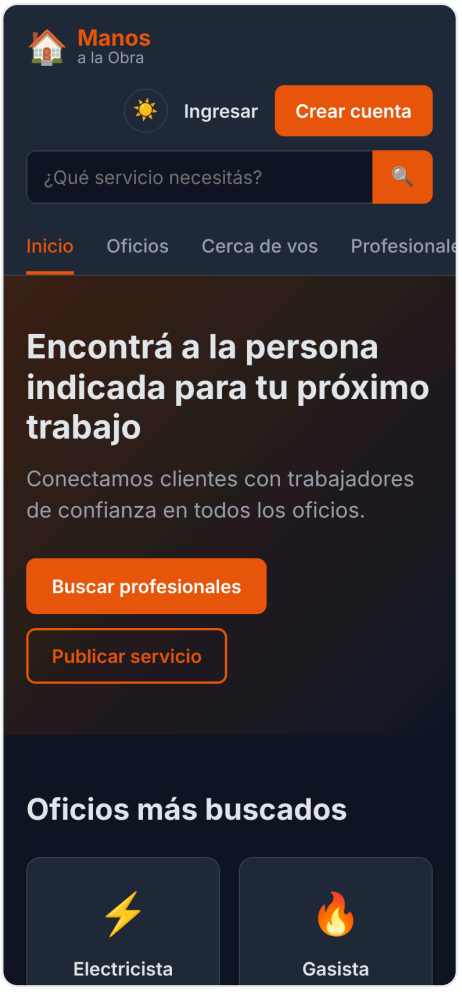
Variables y modo oscuro

Los colores se definen **una sola vez** en `:root` y se usan con `var(--nombre)`. El modo oscuro no repite estilos: **solo cambia el valor de las variables** cuando el `<body>` tiene la clase `oscuro`, y como todo el sitio usa `var(--...)`, se actualiza solo.

```css
:root {
    --primario: #e8590c;      /* naranja: botones y detalles */
    --texto: #1f2937;         /* casi negro */
    --fondo: #f6f7f9;         /* gris muy claro */
    --tarjeta: #ffffff;       /* fondo de tarjetas y cajas */
}
body.oscuro {
    --texto: #e5e7eb;         /* casi blanco */
    --fondo: #111827;         /* casi negro */
    --tarjeta: #1f2937;
    color-scheme: dark;       /* el navegador también oscurece calendarios y listas */
}
.profesional-card { background: var(--tarjeta); }   /* blanco o gris oscuro, según el modo */
```



Celular, modo claro.



Celular, modo oscuro: mismo CSS, otras variables.

Los colores en **hexadecimal** (`#e8590c`) indican cuánto rojo, verde y azul tienen. `rgba(0, 0, 0, 0.06)` es negro con 6% de opacidad, para sombras suaves.

Especificidad: cuando dos reglas chocan

Si dos reglas cambian la misma propiedad, gana la más **específica**; si pesan lo mismo, gana la que está más abajo en el archivo. Ejemplo real: `.profesional-card` tiene borde gris y está **después** de la regla del plan Destacado. Si la regla fuera solo `.destacada` , ganaría el gris. Por eso se escribe con dos clases, que pesan más que una:

```
.profesional-card.destacada,
.perfil-caja.destacada { border: 2px solid var(--primario); }
```

Lo mismo pasa con `body.oscuro .mensaje-error`, que le gana a `.mensaje-error` para cambiar el color en modo oscuro.

El modelo de caja



`* { box-sizing: border-box; }` hace que el ancho de un elemento incluya su padding y su borde: si una caja mide 300px, mide 300px en total.

Unidades de medida

Unidad	Qué significa	Dónde se usa
`px`	Píxeles: tamaño fijo.	Bordes, espacios, redondeos.
`rem`	Relativo a la letra base (16px). `1.6rem` ≈ 25px.	Tamaños de texto.
`%`	Porcentaje del elemento que lo contiene.	`width: 100%`, `border-radius: 50%` (círculo).
`vh`	Porcentaje del alto de la ventana.	`min-height: 100vh` en formularios.
`fr`	Fracción del espacio libre en un grid.	Columnas de tarjetas.

Flexbox: elementos en fila

```
.barra-superior { display: flex; align-items: center; gap: 20px; }  /* logo, buscador y botones en fila */
.acciones { margin-left: auto; }              /* empuja los botones hasta la derecha */
.formulario .fila { display: flex; gap: 16px; }   /* Nombre y Apellido lado a lado */
.formulario .fila > label { flex: 1 1 0; }      /* las dos columnas del mismo ancho */
.seccion-encabezado { display: flex; justify-content: space-between; }  /* título y "Ver todos" */
```

Grid: tarjetas que se acomodan solas

```
.profesionales-grid {
    display: grid;
    grid-template-columns: repeat(auto-fill, minmax(260px, 1fr));
    gap: 20px;
}
```

"Poné todas las columnas que entren, cada una de al menos 260px, y repartí el espacio sobrante." En computadora entran 3 tarjetas por fila y en celular, 1, sin escribir nada especial para cada pantalla.

Encabezado fijo

```
.header-principal { position: sticky; top: 0; z-index: 10; }  /* queda pegado arriba y por encima */
.seccion-titulo { scroll-margin-top: 130px; }  /* al saltar con #profesionales, el encabezado no tapa el título */
```

Estados y transiciones

```
.boton-lleno:hover { background: var(--primario-oscuro); }   /* se oscurece con el mouse */
.oficio-card:hover { transform: translateY(-3px); }        /* la tarjeta "sube" */
.formulario input:focus {                                  /* el campo donde se escribe */
    outline: none;
    border-color: var(--primario);
    box-shadow: 0 0 0 3px rgba(232, 89, 12, 0.15);           /* halo naranja */
}
```

`transition: background 0.2s` hace que esos cambios sean suaves en vez de instantáneos.

Pasos numerados con CSS puro

En "Cómo funciona", los números dentro de círculos naranjas no están escritos en el HTML: los pone el CSS.

```
.pasos { list-style: none; counter-reset: paso; }        /* empieza a contar desde 0 */
.pasos li { counter-increment: paso; position: relative; padding-left: 44px; }  /* +1 por cada paso */
.pasos li::before {
    content: counter(paso);        /* escribe el número */
    position: absolute; left: 0;   /* a la izquierda del texto */
    width: 30px; height: 30px; border-radius: 50%;   /* círculo */
    background: var(--primario); color: var(--blanco);
}
```

Mensajes de error y de éxito

```
.mensaje-error { background: #fdecea; color: #a12622; border-left: 4px solid #dc2626; }
.mensaje-ok    { background: #e7f6ec; color: #1d6b3a; border-left: 4px solid #16a34a; }
```

PHP decide **si** se muestra un mensaje; el CSS decide **cómo** se ve.

Media query: diseño para celular

Las reglas dentro de `@media` solo se aplican cuando la pantalla mide 760px o menos. Necesitan la etiqueta `<meta name="viewport">` del `<head>`.

```
@media (max-width: 760px) {
    .buscador { order: 3; flex-basis: 100%; }          /* el buscador baja a otra línea */
    .hero h1 { font-size: 1.8rem; }                    /* título más chico */
    .formulario .fila { flex-direction: column; }      /* campos uno debajo del otro */
    .perfil-cabecera { flex-direction: column; text-align: center; }
    .perfil-datos, .pie-confianza ul { grid-template-columns: 1fr; }  /* una sola columna */
}
```

> Si se cambia el nombre de una clase en el HTML, hay que cambiarlo igual en `style.css`: si no coinciden exactamente (mayúsculas, guiones), el estilo deja de aplicarse.

4 Base de datos

Dónde se guarda la información y cómo se relaciona.

La base se llama `manos_a_la_obra`, usa el juego de caracteres `utf8mb4` (tildes, ñ y emojis) y tiene **cuatro tablas**. Se crea ejecutando `database.sql` en SQLyog.

clientes

- 🔑 **id**
- nombre, apellido
- email (UNIQUE)
- password
- creado_en

resenas

- 🔑 **id**
- → profesional_id
- → cliente_id
- puntaje (1 a 5)
- comentario
- creado_en

profesionales

- 🔑 **id**
- nombre, apellido
- email (UNIQUE), password
- iniciales
- oficio, descripcion
- ubicacion, domicilio
- → barrio_id
- telefono, experiencia
- fecha_alta, fecha_baja
- valoracion, cantidad_resenas
- destacado_hasta

1 cliente → muchas reseñas
resenas.cliente_id apunta a clientes.id

1 profesional → muchas reseñas
resenas.profesional_id apunta a profesionales.id

barrios

- 🔑 **id**
- barrio (UNIQUE)
- zona

1 barrio → muchos profesionales
profesionales.barrio_id apunta a barrios.id

Tabla clientes

Solo los datos de la cuenta: un cliente no publica nada.

Columna	Tipo	Qué guarda
`id`	INT, AUTO_INCREMENT, PK	Número único. MySQL lo asigna solo (1, 2, 3...).
`nombre`, `apellido`	VARCHAR(60)	Texto de hasta 60 caracteres.
`email`	VARCHAR(120), UNIQUE	No puede haber dos clientes con el mismo email.
`password`	VARCHAR(255)	La contraseña **encriptada** (hash), nunca la original.
`creado_en`	TIMESTAMP	Fecha y hora de creación; se completa sola.

Tabla profesionales

Columna	Tipo	Qué guarda
Cuenta (se carga al registrarse)		
`id`	INT, AUTO_INCREMENT, PK	Número único.
`nombre`, `apellido`, `email`, `password`	igual que en clientes	Datos de la cuenta.
`iniciales`	VARCHAR(5)	Se calculan al registrarse (Ana López → AL). Van en el círculo de la tarjeta.
Perfil (NULL hasta completar "Postular servicio")		
`oficio`	VARCHAR(60), NULL	Mientras esté vacío, el profesional no aparece en el inicio.
`descripcion`	VARCHAR(300), NULL	Qué trabajos hace.
`ubicacion`	VARCHAR(100), NULL	La ciudad.
`domicilio`	VARCHAR(120), NULL	Calle y número. **No se muestra** en la página.
`barrio_id`	INT, NULL, FK → barrios	En qué barrio trabaja.
`telefono`	VARCHAR(30), NULL	Contacto.
`experiencia`	INT, NULL	Años de experiencia (0 a 60).
`fecha_alta`, `fecha_baja`	DATE, NULL	Desde cuándo trabaja. Con una fecha de baja pasada, deja de aparecer.
Calculados y plan		
`valoracion`	DECIMAL(2,1)	Promedio de las reseñas con un decimal (4.5).
`cantidad_resenas`	INT	Cuántas reseñas recibió.
`destacado_hasta`	DATE, NULL	Hasta qué día tiene el plan Destacado. NULL = no lo tiene.

Tabla resenas

Cada fila es la opinión de **un** cliente sobre **un** profesional. Es la tabla que une a las otras dos.

```sql
CREATE TABLE resenas (
    id INT AUTO_INCREMENT PRIMARY KEY,
    profesional_id INT NOT NULL,
    cliente_id INT NOT NULL,
    puntaje TINYINT NOT NULL,                -- de 1 a 5 estrellas
    comentario VARCHAR(300) NOT NULL,
    creado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (profesional_id) REFERENCES profesionales(id) ON DELETE CASCADE,
    FOREIGN KEY (cliente_id) REFERENCES clientes(id) ON DELETE CASCADE,
    UNIQUE (profesional_id, cliente_id)
);
```

Tabla barrios

Tabla de apoyo con 19 barrios de Rosario, cada uno con su zona (Centro, Norte, Noroeste, Oeste, Sur). La usan la lista de barrios de "Profesionales cerca" y la de `postular.php`. Tenerlos en una tabla (en vez de escribirlos en cada profesional) evita errores de tipeo y permite agrupar por zona.

Conceptos que aparecen en las tablas

Concepto	Qué significa en este proyecto
PRIMARY KEY (PK)	La columna que identifica cada fila sin repetirse: `id`. Con **AUTO_INCREMENT**, MySQL pone el número siguiente.
FOREIGN KEY (FK)	Una columna que guarda la PK de otra tabla. MySQL controla que ese valor exista: no se puede cargar una reseña para el profesional 99 si no existe.
ON DELETE CASCADE	Si se borra un profesional o un cliente, se borran solas sus reseñas. No quedan reseñas "huérfanas".
UNIQUE	El valor no se repite: emails, nombres de barrio. `UNIQUE (profesional_id, cliente_id)` impide que la **combinación** se repita: un cliente, una reseña por profesional.
NULL	"Sin valor todavía". Las columnas del perfil empiezan en NULL; `fecha_baja` NULL = sigue trabajando; `destacado_hasta` NULL = sin plan.
Relación 1 a muchos (1:N)	Un profesional tiene muchas reseñas; cada reseña es de un solo profesional. Igual con clientes y con barrios.

Decisiones de diseño (para defender)

P: ¿Por qué clientes y profesionales en tablas separadas?
Porque guardan datos distintos: el profesional tiene todo un perfil que el cliente no necesita. Con una sola tabla, el cliente tendría muchas columnas siempre vacías. La contra: el login y el control de email repetido buscan en las dos tablas con `UNION`.

P: ¿Por qué guardar `valoracion` si se puede calcular con AVG?
Para que el inicio ordene por valoración con una consulta simple y rápida. A cambio, hay que recalcularla cada vez que entra una reseña (lo hace `perfil.php`).

P: ¿Por qué `destacado_hasta` es una fecha y no un sí/no?
Porque el plan es mensual: con una fecha, el plan **vence solo** cuando pasa ese día, sin que nadie tenga que desactivarlo.

P: Un mismo email, ¿puede estar en las dos tablas?
`UNIQUE` controla dentro de una tabla, no entre tablas. Ese control lo hace `guardar.php` buscando el email en ambas antes de crear la cuenta. Si no, al ingresar no se sabría qué cuenta abrir.

Datos de ejemplo

Todas las cuentas tienen la contraseña **123456** (en la base se ve el hash que empieza con `$2y$10$`).

Tipo	Email	Datos
Profesional	`juan@ejemplo.com`	Electricista, Centro · 2 reseñas → 4.5
Profesional	`maria@ejemplo.com`	Pintor, Fisherton · 1 reseña → 5.0
Profesional	`carlos@ejemplo.com`	Plomero, Echesortu · 1 reseña → 4.0 · **plan Destacado** por un mes
Profesional	`ana@ejemplo.com`	Gasista, Ludueña · sin reseñas
Profesional	`diego@ejemplo.com`	Carpintero, Arroyito · sin reseñas
Cliente	`lucia@ejemplo.com`	Opinó sobre Juan y María.
Cliente	`martin@ejemplo.com`	Opinó sobre Juan y Carlos.

Al final del script, un `UPDATE` calcula `valoracion` y `cantidad_resenas` a partir de las reseñas cargadas. El `barrio_id` de cada profesional se busca por nombre con una subconsulta: `(SELECT id FROM barrios WHERE barrio = 'Centro')`.

> **El script borra las tablas si ya existían** (`DROP TABLE`): al volver a ejecutarlo se pierde todo lo creado. Se borran en orden (`resenas` primero, `barrios` al final) porque MySQL no deja borrar una tabla mientras otra tenga claves foráneas hacia ella.

5 Conexión entre PHP y la base

Cómo las páginas leen y guardan datos, archivo por archivo.

config.php: abrir la conexión

```php
$servidor = "localhost";       // MySQL está en la misma computadora
$usuario  = "root";            // mismo usuario que en SQLyog
$clave    = "";                // XAMPP viene sin contraseña
$base     = "manos_a_la_obra";

try {
    $conexion = new mysqli($servidor, $usuario, $clave, $base);
} catch (mysqli_sql_exception $e) {
    die("No se pudo conectar a la base de datos. Revisá que MySQL esté prendido en XAMPP "
        . "y que hayas ejecutado database.sql. Detalle: " . $e->getMessage());
}
$conexion->set_charset("utf8mb4");   // tildes y ñ
```

Desde PHP 8.1, si mysqli no puede conectarse, **lanza una excepción** (corta todo con un error técnico). Con `try / catch` se ataja y se muestra un mensaje claro. Las demás páginas empiezan con `require "config.php";` y usan `$conexion`.

> SQLyog y la página no se conectan entre sí: los dos se conectan **al mismo servidor MySQL**. Por eso lo que se edita en SQLyog aparece en la página al recargar.

Consultas preparadas: la regla de oro

Cuando una consulta lleva datos que escribió el usuario, **nunca se pegan directo en el SQL**. Se ponen signos `?` y los datos se pasan aparte con `bind_param`. Así MySQL los trata siempre como datos y nunca como código: esto evita la **inyección SQL**.

```php
$consulta = $conexion->prepare("SELECT oficio FROM profesionales WHERE id = ?");  // 1. preparar
$consulta->bind_param("i", $usuario_id);                                          // 2. completar los ?
$consulta->execute();                                                             // 3. ejecutar
$fila = $consulta->get_result()->fetch_assoc();                                   // 4. leer el resultado
```

La primera parte de `bind_param` indica el tipo de cada `?`, en orden: `s` = texto, `i` = número entero. Tiene que haber una letra por cada `?`. `fetch_assoc()` devuelve una fila por vez como array (`$fila["oficio"]`); `fetch_all(MYSQLI_ASSOC)` devuelve todas juntas.

Sesiones y cookies

`session_start()` va al principio de cada página. Al registrarse o ingresar se guardan estos datos, disponibles en todas las páginas hasta tocar "Salir":

```
$_SESSION["usuario_id"]        // el id de la fila en SU tabla
$_SESSION["usuario_nombre"]
$_SESSION["usuario_apellido"]
$_SESSION["usuario_tipo"]      // "cliente" o "profesional": dice en qué tabla está ese id
```

> Cada tabla numera sus filas por separado: puede existir el cliente 1 y el profesional 1. Por eso el id solo no alcanza: siempre va con `usuario_tipo`.

La **sesión** se guarda en el servidor y dura hasta cerrar sesión o el navegador. La **cookie** se guarda en el navegador del usuario: el modo oscuro usa una cookie `tema` que dura un año, para que la próxima vez el sitio ya aparezca con el modo elegido.

Registro: registro.php → guardar.php

1. Validar

Que no falte nada, que el tipo sea "cliente" o "profesional", que el email sea válido (`filter_var`), que la contraseña tenga al menos 6 caracteres y que nombre, apellido y email no superen el largo de sus columnas (60 y 120). Si algo falla, vuelve con un código: `registro.php?error=existe`.

2. Revisar que el email no exista en ninguna de las dos tablas

```
"SELECT id FROM clientes WHERE email = ?
 UNION
 SELECT id FROM profesionales WHERE email = ?"
```

`UNION` junta los resultados de dos consultas. Si aparece alguna fila, el email ya está usado.

3. Encriptar la contraseña y guardar en la tabla que corresponde

```
$hash = password_hash($password, PASSWORD_DEFAULT);   // "123456" → "$2y$10$ioq8..."

if ($tipo === "profesional") {
    $iniciales = mb_strtoupper(mb_substr($nombre, 0, 1) . mb_substr($apellido, 0, 1));
    $insertar = $conexion->prepare(
        "INSERT INTO profesionales (nombre, apellido, email, password, iniciales) VALUES (?, ?,
?, ?, ?)");
    $insertar->bind_param("sssss", $nombre, $apellido, $email, $hash, $iniciales);
} else {
    $insertar = $conexion->prepare(
        "INSERT INTO clientes (nombre, apellido, email, password) VALUES (?, ?, ?, ?)");
    $insertar->bind_param("ssss", $nombre, $apellido, $email, $hash);
}
$insertar->execute();
```

4. Dejarlo logueado

`session_regenerate_id(true)` le da un número de sesión nuevo (si alguien conocía el viejo, no le sirve), se guardan los datos en `$_SESSION` (el id sale de `$conexion->insert_id`) y vuelve al inicio.

Ingresar: login.php

```
SELECT id, nombre, apellido, password, 'cliente'     AS tipo FROM clientes      WHERE email = ?
UNION
SELECT id, nombre, apellido, password, 'profesional' AS tipo FROM profesionales WHERE email = ?
```

Busca el email en las dos tablas. La columna `tipo` es "inventada" en la consulta para saber de qué tabla vino la fila. Después, `password_verify($password, $usuario["password"])` compara lo que escribió con el hash. Si está mal el email o la contraseña, el mensaje es el mismo a propósito: así nadie puede averiguar qué emails tienen cuenta.

`logout.php` hace `session_unset()` (vacía los datos) y `session_destroy()` (elimina la sesión) y vuelve al inicio.

Completar el perfil: postular.php

Al principio controla la sesión: sin sesión → `login.php`; si es cliente → inicio. Valida todo (oficio de la lista, barrio que exista, largos máximos, experiencia entre 0 y 60) y las fechas con esta función:

```
function fecha_valida($fecha) {
    $fecha_armada = DateTime::createFromFormat("Y-m-d", $fecha);
    return $fecha_armada && $fecha_armada->format("Y-m-d") === $fecha;
}
```

Arma una fecha a partir del texto y la vuelve a escribir: si no da lo mismo, el texto no era una fecha real (rechaza "hola" o "2024-02-31"). Además, la fecha de alta no puede ser futura ni la de baja anterior a la de alta. Como la fila ya existe desde el registro, guardar es siempre un **UPDATE**:

```
UPDATE profesionales
SET oficio = ?, descripcion = ?, ubicacion = ?, domicilio = ?, barrio_id = ?,
    telefono = ?, experiencia = ?, fecha_alta = ?, fecha_baja = ?
WHERE id = ?          -- bind_param("ssssisissi", ...): 10 signos, 10 letras
```

Mostrar y buscar profesionales: index.php

La consulta se **arma por partes**: siempre trae a los de perfil completo y en actividad; si hay oficio elegido o búsqueda, se agrega esa condición.

```
$sql = "SELECT profesionales.*, barrios.barrio,
               (destacado_hasta IS NOT NULL AND destacado_hasta ⩾ CURDATE()) AS destacado
        FROM profesionales
        LEFT JOIN barrios ON barrios.id = profesionales.barrio_id
        WHERE oficio IS NOT NULL
          AND (fecha_baja IS NULL OR fecha_baja ⩾ CURDATE())";
$tipos = "";  $valores = [];

if ($oficio ≠= "") {
    $sql .= " AND oficio = ?";   $tipos .= "s";   $valores[] = $oficio;
}
if ($buscar ≠= "") {
    $sql .= " AND (oficio LIKE ? OR nombre LIKE ? OR apellido LIKE ? OR descripcion LIKE ?
              OR ubicacion LIKE ? OR barrios.barrio LIKE ?)";
    $patron = "%" . $buscar . "%";
    $tipos .= "ssssss";
    array_push($valores, $patron, $patron, $patron, $patron, $patron, $patron);
}
$sql .= " ORDER BY destacado DESC, valoracion DESC, cantidad_resenas DESC";

$consulta = $conexion→prepare($sql);
if ($tipos ≠= "") { $consulta→bind_param($tipos, ...$valores); }
$consulta→execute();
```

Parte	Qué hace
`LEFT JOIN barrios`	Trae el nombre del barrio. Es LEFT para no dejar afuera a los profesionales sin barrio.
`... AS destacado`	Columna calculada: 1 si tiene el plan vigente, 0 si no. `CURDATE()` es la fecha de hoy.
`oficio IS NOT NULL`	Deja afuera a los que no completaron el perfil (no hay tarjetas vacías).
`fecha_baja ...`	Deja afuera a los que ya no trabajan.
`.=`	Agrega texto al final de `$sql`: la consulta crece solo con lo que hace falta.
`LIKE '%pint%'`	Busca el texto en cualquier parte: "pint" encuentra "Pintor". `%` = cualquier cosa.
`...$valores`	"Desarma" el array y pasa cada elemento como un dato aparte.
`ORDER BY destacado DESC, valoracion DESC`	Primero los destacados (1 antes que 0) y después los mejor valorados.

Profesionales cerca

```
SELECT profesionales.*, barrios.barrio, (...) AS destacado
FROM profesionales
JOIN barrios ON barrios.id = profesionales.barrio_id
WHERE oficio IS NOT NULL AND (fecha_baja IS NULL OR fecha_baja ⩾ CURDATE())
  AND barrios.zona = ?                                    -- solo la misma zona
ORDER BY destacado DESC, (barrios.id = ?) DESC, valoracion DESC
LIMIT 6                                                   -- como mucho seis
```

`(barrios.id = ?)` da 1 si es el mismo barrio elegido y 0 si no: con `DESC`, los del mismo barrio quedan antes que los del resto de la zona. Cada tarjeta lleva la etiqueta "🏠 En tu barrio" o "📍 Zona Centro".

El perfil y las reseñas: perfil.php

1. **Leer el id:** `$id = (int) ($_GET["id"] ?? 0);` → `(int)` convierte cualquier cosa en número; un texto queda 0.
2. **Buscar al profesional primero** (mismas condiciones que el inicio). Si no existe o está de baja, no se acepta ninguna reseña y se muestra "Perfil no disponible".
3. **Guardar la reseña** si quien envía es cliente, el puntaje está entre 1 y 5, hay comentario de hasta 300 caracteres y no opinó antes (un `SELECT` en `resenas`; la base igual lo impediría por el `UNIQUE`).
4. **Recalcular la valoración** con subconsultas:

```
UPDATE profesionales
SET valoracion       = (SELECT ROUND(AVG(puntaje), 1) FROM resenas WHERE profesional_id = ?),
    cantidad_resenas = (SELECT COUNT(*)               FROM resenas WHERE profesional_id = ?)
WHERE id = ?
```

5. **Redirigir:** `header("Location: perfil.php?id=3&ok=1#resenas")`. Si el usuario aprieta F5, solo recarga y no reenvía la reseña. Esta técnica se llama **Post/Redirect/Get**.
6. **Mostrar las reseñas** con el nombre de quien las escribió, usando `JOIN`:

```
SELECT resenas.puntaje, resenas.comentario, resenas.creado_en, resenas.cliente_id,
       clientes.nombre, clientes.apellido
FROM resenas
JOIN clientes ON clientes.id = resenas.cliente_id     -- "pega" cada reseña con su cliente
WHERE resenas.profesional_id = ?
ORDER BY resenas.creado_en DESC, resenas.id DESC
```

como_funciona.php

Para el cliente cuenta sus reseñas con `SELECT COUNT(*) AS cantidad FROM resenas WHERE cliente_id = ?`. Para el profesional lee `oficio`, `fecha_baja`, `valoracion`, `cantidad_resenas` y `destacado_hasta` y decide qué mostrar. Las fechas `AAAA-MM-DD` se pueden comparar como texto: `"2026-11-28" ≥ date("Y-m-d")`.

Contacto: contacto.php (sin base de datos, con mail)

1. **Valida** los campos, el email con `filter_var` (que además rechaza saltos de línea: nadie puede inyectar cabeceras de mail) y el asunto contra la lista.
2. **Revisa cada imagen** de `$_FILES["imagenes"]`: que haya subido bien, que pese hasta 2 MB, que sean como mucho 3, y con `getimagesize()` que sea **una imagen de verdad** (no alcanza con la extensión: cualquiera puede renombrar un archivo).
3. **Arma el mail "multipart"**: una parte con el texto y una parte por imagen, separadas por un "límite". Las imágenes viajan convertidas a texto con `base64_encode`, porque un mail solo lleva texto. La cabecera `Reply-To` hace que al responder le llegue a quien escribió.
4. **Envía** con `mail($correo_destino, $titulo, $cuerpo, $cabeceras)` y redirige con `?enviado=1` (Post/Redirect/Get).

```php
// Con name="imagenes[]" multiple, $_FILES trae una lista por cada dato:
$_FILES["imagenes"]["name"][0]       // nombre de la primera imagen
$_FILES["imagenes"]["tmp_name"][0]   // dónde la guardó PHP temporalmente
$_FILES["imagenes"]["size"][0]       // cuánto pesa, en bytes
$_FILES["imagenes"]["error"][0]      // 0 = subió bien; 4 = no eligió ninguna
```

Modo oscuro: tema.php

```php
$actual = $_COOKIE["tema"] ?? "claro";
$nuevo  = $actual === "oscuro" ? "claro" : "oscuro";
setcookie("tema", $nuevo, time() + 60 * 60 * 24 * 365, "/");   // dura un año
// ... vuelve a la página anterior (HTTP_REFERER), quedándose solo con "pagina.php?..."
```

Cada página lee la cookie (`$modo_oscuro = ($_COOKIE["tema"] ?? "") === "oscuro";`) y, si corresponde, le pone la clase `oscuro` al `<body>`. De la página anterior se queda solo con el nombre del archivo, para no mandar al usuario a otro sitio.

Resumen: qué hace cada archivo con la base

Archivo	Operaciones	Detalle
`index.php`	SELECT + LEFT JOIN / JOIN, LIKE	Profesionales, búsqueda, filtro, cerca, barrios, aviso de perfil.
`perfil.php`	SELECT + JOIN, INSERT, UPDATE con subconsultas	Perfil, reseñas, guardar reseña y recalcular.
`guardar.php`	SELECT con UNION + INSERT	Email libre en las dos tablas y crear la cuenta.
`login.php`	SELECT con UNION	Buscar la cuenta y verificar la contraseña.
`postular.php`	SELECT + UPDATE	Cargar y guardar el perfil.
`como_funciona.php`	SELECT con COUNT	Reseñas del cliente, estado del profesional.
`contacto.php` , `tema.php` , `logout.php`	Ninguna	Mail, cookie y sesión.

Resumen de seguridad

Riesgo	Cómo se evita
Inyección SQL	Consultas preparadas con `?` y `bind_param` en todas las consultas con datos del usuario.
XSS (código HTML/JS en un campo)	Todo lo que viene del usuario o de la base se imprime con `htmlspecialchars()` .
Contraseñas expuestas	`password_hash()` al guardar y `password_verify()` al ingresar.
Robo de sesión	`session_regenerate_id(true)` al iniciar sesión.

Acceso sin permiso	Cada página revisa la sesión en el servidor (no solo oculta botones).
Cabeceras de mail falsas	`filter_var(..., FILTER_VALIDATE_EMAIL)` rechaza saltos de línea.
Archivos que no son imágenes	`getimagesize()` revisa el contenido real del archivo.
Redirección a otro sitio	`tema.php` usa solo el nombre del archivo de la página anterior.

6 Relevamiento y pruebas

Qué se revisó, qué errores aparecieron y cómo se corrigieron.

Antes de la entrega final se revisó **todo el código, archivo por archivo**, y se probó cada página con los tres tipos de usuario (visitante, cliente y profesional), con datos normales y con datos "malos" a propósito: textos muy largos, fechas imposibles, ids que no existen, archivos que no son imágenes. Cada error se confirmó primero, se corrigió y se volvió a probar.

Errores que cortaban la página (error 500)

Desde PHP 8.1, cuando MySQL rechaza una consulta, PHP corta la página entera. Estos casos se daban con datos que el formulario no frenaba:

	Error	Causa	Corrección
Alta	Reseña a un profesional que no existe (`perfil.php?id=99`)	Se intentaba guardar antes de buscar al profesional: fallaba la FOREIGN KEY.	Ahora se busca al profesional **antes** y, si no existe, no se guarda nada.
Alta	Nombre, apellido o email más largos que la columna	El HTML no tenía `maxlength` y PHP no controlaba el largo: "Data too long".	`maxlength` en el formulario y control con `mb_strlen` en `guardar.php` .
Alta	Fecha inválida en el perfil ("hola", 31 de febrero)	Se mandaba a MySQL sin revisar: "Incorrect date value".	Función `fecha_valida()` en `postular.php` .
Alta	Ciudad, domicilio o teléfono demasiado largos	Igual que el nombre.	`maxlength` + control en PHP con los largos de las columnas.
Media	Campo de imágenes enviado sin `[]`	El `for` esperaba una lista.	Se revisa con `is_array` antes de recorrerla.
Media	MySQL apagado: error técnico largo	El `if ($conexion->connect_error)` ya no se ejecutaba en PHP 8.	`try / catch` en `config.php` con un mensaje claro.

Otros problemas corregidos

	Problema	Corrección
Media	Se podía dejar una reseña a un profesional dado de baja.	Mismo arreglo que el id inexistente: solo se aceptan reseñas de profesionales activos.
Media	El número de sesión no cambiaba al iniciar sesión.	`session_regenerate_id(true)` en `login.php` y `guardar.php` .
Baja	Se aceptaba una fecha de alta futura.	Control en PHP y `max` en el calendario.
Baja	Si la reseña tenía un error, se perdía el puntaje elegido.	El `<select>` vuelve a marcar el puntaje.

Baja	"1 reseñas" en plural.	Singular o plural según la cantidad.
Baja	El pie decía "Perfiles verificados", algo que el sitio no hace.	Cambiado por "Profesionales de tu zona".

Mejoras de legibilidad

- Cada archivo PHP empieza con un comentario que explica qué hace.
- `index.php` tiene un índice de sus 7 partes y cada bloque de PHP dice a qué sección corresponde.
- `style.css` tiene un índice de 16 partes numeradas.
- `perfil.php` numera sus pasos: 1. datos del profesional, 2. guardar reseña, 3. reseñas.
- Se sacaron comentarios que hablaban de versiones viejas ("antes era registro.html").

Pruebas realizadas

Prueba	Resultado esperado	Estado
Todas las páginas como visitante, cliente y profesional	Cargan sin errores	✓
Registrarse como cliente y como profesional (con tildes y ñ)	Cuenta creada, sesión iniciada, iniciales bien calculadas	✓
Email repetido / contraseña corta / nombre de 80 letras	Mensaje de error, no se crea la cuenta	✓
Buscar "pint", filtrar "Plomero", elegir barrio Centro	Resultados correctos, destacado primero	✓
Dejar reseña de 2 estrellas a Carlos (4.0)	Valoración 3.0 con 2 reseñas	✓
Segunda reseña al mismo profesional	"Ya dejaste tu reseña"	✓
Reseña a id inexistente / profesional dado de baja	"Perfil no disponible", no se guarda	✓
Perfil con fecha "hola", "2024-02-31", futura o teléfono de 40 dígitos	Mensaje de error claro	✓
Contacto con 2 imágenes / 4 imágenes / archivo falso	Mail con adjuntos idénticos / error / error	✓
Plan Destacado vigente y vencido	Primero con insignia / vuelve a su lugar	✓
Intento de inyección SQL en el buscador (`' OR 1=1 --`)	Sin resultados, sin error	✓
Modo oscuro y vista de celular	Todo legible	✓

7 Puesta en marcha

Pasos para que todo funcione y qué revisar si algo falla.

Pasos

1. Tener los archivos en `C:\xampp\htdocs\manos_a_la_obra_con_css\` (con Git: `git pull` para traer lo último).
2. En el panel de XAMPP, prender **Apache** y **MySQL**.
3. En SQLyog, conectarse a `localhost`, usuario `root`, contraseña vacía.
4. Abrir `database.sql` y ejecutarlo completo con **Shift+F9** (F9 ejecuta una sola consulta). Al refrescar (F5) tienen que aparecer `barrios`, `clientes`, `profesionales` y `resenas`.
5. Entrar a `http://localhost/manos_a_la_obra_con_css/index.php`, siempre por el navegador, nunca con doble clic.

Envío de mails (formulario de contacto)

XAMPP no manda mails hasta configurar una cuenta. Se hace una vez por computadora:

1. En la cuenta iseiclaude@gmail.com: activar la **verificación en 2 pasos** y crear una **contraseña de aplicación** en `myaccount.google.com/apppasswords` (16 letras; no es la contraseña normal y no se sube a GitHub).
2. En `C:\xampp\sendmail\sendmail.ini`: `smtp_server=smtp.gmail.com`, `smtp_port=587`, `smtp_ssl=tls`, `auth_username` y `force_sender` con el email, `auth_password` con la clave sin espacios.
3. En `php.ini` (XAMPP → Config de Apache → PHP): `sendmail_path = C:\xampp\sendmail\sendmail.exe -t`. Comprobar con `C:\xampp\php\php.exe -i | findstr sendmail_path`.
4. Reiniciar Apache. Si falla, el detalle está en `C:\xampp\sendmail\error.log`.

Activar el plan Destacado

```
-- Activar por un mes desde hoy
UPDATE profesionales SET destacado_hasta = DATE_ADD(CURDATE(), INTERVAL 1 MONTH) WHERE email =
'juan@ejemplo.com';
-- Sacarlo
UPDATE profesionales SET destacado_hasta = NULL WHERE email = 'juan@ejemplo.com';
```

Errores comunes

Lo que aparece	Qué revisar
"No se pudo conectar a la base de datos... Unknown database"	No se ejecutó `database.sql`.
"No se pudo conectar... No connection could be made"	MySQL apagado en XAMPP.

"No se pudo conectar... Access denied for user 'root'"	Usuario o contraseña de `config.php` distintos a los de SQLyog.
`Table ... doesn't exist` o `Unknown column`	Se actualizaron los `.php` pero no la base. Volver a ejecutar `database.sql` con Shift+F9.
Se ve el código PHP o se descarga el archivo	Se abrió con doble clic: entrar por `http://localhost/...`.
"Not Found" (404)	El archivo no está en la carpeta (falta `git pull`) o la dirección no coincide con el nombre de la carpeta.
La página no carga	Apache apagado en XAMPP.
Tildes con símbolos raros (MarÃa)	Falta `set_charset("utf8mb4")` o la base no está en utf8mb4.
La página se ve sin estilos	`style.css` no está en la misma carpeta.
"No pudimos enviar el mensaje" en Contacto	Falta configurar el envío de mails; ver `error.log`.

8 Preparar la defensa

Guion, preguntas probables por parte, glosario y lista de repaso.

El proyecto en un minuto

> "Manos a la Obra es un sitio web que conecta a vecinos de Rosario con profesionales de oficios. Los profesionales publican su perfil con su oficio, barrio y contacto; los clientes buscan por oficio, por palabra o por cercanía, leen las reseñas de otros clientes y dejan la suya. Está hecho con HTML y CSS para la parte visual, PHP para la lógica del servidor y MySQL como base de datos, sobre XAMPP. La base tiene cuatro tablas: clientes, profesionales, reseñas y barrios. Publicar es gratis, y el sitio se monetiza con un plan Destacado mensual para los profesionales que quieren más visibilidad."

Guion para la demostración

1. **Inicio:** buscador ("pint"), filtro por oficio, "Profesionales cerca" con un barrio.
2. **Perfil:** entrar a un profesional y mostrar sus reseñas.
3. **Cliente:** ingresar como Lucía, dejar una reseña a Carlos y mostrar que cambió la valoración.
4. **SQLyog:** mostrar la fila nueva en `resenas` y la valoración actualizada en `profesionales`.
5. **Plan Destacado:** activarle el plan a Juan con un `UPDATE`, recargar el inicio y ver que sube al primer lugar.
6. **Profesional nuevo:** crear una cuenta, ver el aviso, completar el perfil y verlo en el inicio.
7. **Cómo funciona** con los tres usuarios, **modo oscuro** y **vista de celular** (F12 → ícono de celular).

> **Antes de presentar:** ejecutar `database.sql` para arrancar con los datos limpios, prender XAMPP y ensayar el guion completo una vez. Si en un ensayo Lucía ya opinó sobre Carlos, en la defensa no va a poder hacerlo de nuevo.

Preguntas probables

Generales y de análisis

P: ¿Qué problema resuelve el sistema?
Encontrar un trabajador de confianza suele depender del boca en boca. El sitio reúne a los profesionales con su zona, contacto, experiencia y opiniones de otros clientes, para comparar antes de llamar.

P: ¿Quiénes son los usuarios y qué puede hacer cada uno?
Visitante: buscar y ver perfiles. Cliente: además, dejar una reseña por profesional. Profesional: publicar y editar su perfil, y contratar el plan Destacado. El sitio muestra opciones distintas a cada uno y lo controla en el servidor.

P: ¿Cómo se monetiza?
Modelo freemium: publicar es gratis para que el sitio se llene de profesionales, y quien quiere más visibilidad paga el plan Destacado mensual (aparece primero, con insignia). Se descartó cobrar comisión porque el contacto es directo por teléfono y no hay forma de saber si se hizo el trabajo. El plan no compra reseñas: la valoración la ponen solo los clientes.

P: ¿Cómo probaron el sistema?
Hicimos un relevamiento de todo el código y probamos cada página con los tres usuarios, con datos normales y con datos inválidos a propósito. Encontramos errores (por ejemplo, fechas inválidas o textos más largos que la columna cortaban la página), los corregimos y volvimos a probar (sección 6).

P: ¿Qué mejorarían?
Pagos en línea para el plan Destacado, foto del profesional, recuperar contraseña por mail, que el cliente pueda editar su reseña, un panel de administrador para moderar reseñas y activar planes, y verificación de matrícula para gasistas y electricistas.

HTML

P: ¿Por qué los archivos son .php y no .html?
Porque mezclan HTML con PHP: el servidor ejecuta el PHP (busca datos, decide qué mostrar) y al navegador le llega HTML común.

P: ¿Diferencia entre GET y POST?
GET pone los datos en la dirección: sirve para búsquedas y filtros, que se pueden compartir. POST los manda ocultos: para guardar datos o mandar contraseñas.

P: ¿Para qué sirve `enctype="multipart/form-data"`?
Es obligatorio para mandar archivos en un formulario. Sin eso solo viajan los nombres de los archivos.

P: ¿Por qué validan en el formulario y también en PHP?
Las validaciones del navegador (`required` , `maxlength`) ayudan al usuario pero se pueden saltear. Las de PHP protegen el sistema, porque corren en el servidor.

P: ¿Qué hace `include "tarjeta_profesional.php"`?
Inserta el HTML de la tarjeta en ese lugar. Así la tarjeta se escribe una sola vez y se usa en las dos grillas del inicio.

CSS

P: ¿Cómo funciona el modo oscuro?
Un link lleva a `tema.php`, que guarda la elección en una cookie. Cada página lee la cookie y le pone la clase `oscuro` al body. En el CSS, `body.oscuro` solo cambia el valor de las variables de color, y como todo usa `var(--...)`, se actualiza el sitio entero.

P: ¿Cómo se adapta al celular?
Con la etiqueta viewport, grids con `repeat(auto-fill, minmax(260px, 1fr))` que acomodan las tarjetas solas, y un `@media (max-width: 760px)` que baja el buscador de línea y apila los campos.

P: ¿Flexbox o grid?
Flexbox acomoda en una dirección (fila o columna): encabezado, filas de formularios. Grid acomoda en filas y columnas a la vez: tarjetas de oficios y profesionales.

P: ¿Qué es la especificidad?
La regla que decide qué estilo gana cuando dos reglas chocan: gana la más específica. Por eso el borde del plan Destacado se escribe `.profesional-card.destacada`: dos clases pesan más que una.

Base de datos

P: ¿Cómo se relacionan las tablas?
`resenas` une a clientes y profesionales con dos claves foráneas (uno a muchos con cada una). `profesionales.barrio_id` apunta a `barrios` (un barrio, muchos profesionales).

P: ¿Qué es una clave primaria y una foránea?
La primaria identifica cada fila sin repetirse. La foránea guarda la primaria de otra tabla, y MySQL controla que ese valor exista.

P: ¿Cómo evitan reseñas repetidas?
En la base, con `UNIQUE (profesional_id, cliente_id)`. En PHP, se busca antes y se muestra "Ya dejaste tu reseña", y si ya opinó no se muestra el formulario.

P: ¿Qué pasa si se borra un profesional?
Por `ON DELETE CASCADE`, MySQL borra solas sus reseñas.

P: ¿Cómo funciona "Profesionales cerca"?
Cada barrio tiene una zona en la tabla `barrios`. Se traen los profesionales de la misma zona del barrio elegido, ordenados con los destacados primero y después los del mismo barrio (`ORDER BY (barrios.id = ?) DESC`), hasta seis.

PHP, conexión y seguridad

P: ¿Qué es la inyección SQL y cómo la evitan?
Es escribir código SQL en un campo (`' OR 1=1 --`) para cambiar la consulta. Se evita con consultas preparadas: el SQL lleva `?` y los datos se pasan aparte con `bind_param`, así MySQL nunca los ejecuta como código.

P: ¿Cómo guardan las contraseñas?
Encriptadas con `password_hash()`, que no se puede revertir. Al ingresar, `password_verify()` compara. Nadie con acceso a la base ve las contraseñas reales.

P: ¿Para qué sirve `htmlspecialchars`?
Convierte `<` y `>` en texto antes de mostrarlo. Si alguien escribe `<script>` en una reseña, se ve como texto y no se ejecuta (ataque XSS).

P: ¿Qué es una sesión y por qué guardan `usuario_tipo`?
La sesión permite que el servidor recuerde quién está logueado entre páginas. El tipo dice en qué tabla buscar el id, porque cada tabla numera sus filas por separado.

P: ¿Qué es Post/Redirect/Get?
Después de guardar (POST), PHP redirige con `header("Location: ...")` a la misma página por GET. Si el usuario aprieta F5, solo recarga y no reenvía el formulario. Lo usan la reseña y el contacto.

P: ¿Cómo funciona el buscador por dentro?
Lo escrito llega por `$_GET["buscar"]`; la consulta se arma agregando una condición con `LIKE '%texto%'` sobre oficio, nombre, apellido, descripción, ciudad y barrio, siempre con `?` y `bind_param`.

P: ¿Cómo se manda el mail con imágenes?
Con la función `mail()` de PHP y un mail "multipart": una parte de texto y una por imagen, codificadas en base64. Antes se revisa con `getimagesize()` que cada archivo sea una imagen real.

Glosario

Término	Significado
Servidor / cliente	El servidor (Apache con PHP) arma la página; el cliente (navegador) la muestra.
CRUD	Crear, leer, actualizar y borrar: INSERT, SELECT, UPDATE, DELETE.
PK / FK	Clave primaria / clave foránea.
JOIN / LEFT JOIN	Une filas de dos tablas relacionadas. LEFT conserva las filas de la izquierda aunque no tengan pareja.

UNION	Junta los resultados de dos consultas.
LIKE	Busca texto parecido; `%` = cualquier cantidad de caracteres.
AVG / COUNT / ROUND	Promedio / cantidad de filas / redondeo.
Subconsulta	Una consulta dentro de otra.
Consulta preparada	SQL con `?` y datos aparte. Evita la inyección SQL.
Hash	Versión encriptada e irreversible de una contraseña.
Sesión / cookie	Datos del usuario guardados en el servidor / en el navegador.
Excepción / try-catch	Error que corta el programa / forma de atajarlo y manejarlo.
XSS	Meter código HTML/JS en un campo para que se ejecute en otros navegadores.
Responsive	Diseño que se adapta al tamaño de la pantalla.
Freemium	Uso gratis, con una opción paga que da más beneficios.

Lista para repasar en grupo

- ☐ Explicar las cuatro capas y qué hace cada una.
- ☐ Dibujar las cuatro tablas con sus relaciones en un pizarrón.
- ☐ Recorrer en voz alta el camino de un profesional y de un cliente.
- ☐ Explicar qué pasa desde que se escribe en el buscador hasta que aparecen las tarjetas.
- ☐ Explicar cómo se guarda una reseña y cómo se recalcula la valoración.
- ☐ Explicar consultas preparadas, `password_hash`, `htmlspecialchars` y sesiones.
- ☐ Explicar el modo oscuro (cookie + clase + variables) y el diseño para celular.
- ☐ Contar el relevamiento: qué errores encontramos y cómo los corregimos.
- ☐ Ensayar la demostración completa con la base recién cargada.
- ☐ Repartir las partes: HTML, CSS, base de datos, PHP y seguridad, demostración.